

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Ульяновский государственный технический университет»
Кафедра «Вычислительная техника»

Операционные системы

(название дисциплины)

Лабораторная работа №1

Разработка оболочки

(название работы)

Выполнил

Студент группы ИВТИИБД-21

Мокшин Д.И.

(Фамилия И.О.)

Проверил(а):

ст. преподаватель кафедры «ВТ»

(должность)

Беляев К.С.

(Фамилия И.О.)

Ульяновск
2026

СОДЕРЖАНИЕ

1. Задание на лабораторную работу.....	3
2. Описание реализации.....	3
3. Тестирование.....	7
4. Вывод.....	9
Список использованной литературы.....	10

1. Задание на лабораторную работу

Необходимо создать командную оболочку `myshell`, которая поддерживает встроенные команды: `cd` — для перехода в другой каталог, а при отсутствии аргумента выводит текущий каталог и обновляет переменную среды `PWD`; `clr` — для очистки экрана; `dir` — для вывода содержимого указанного каталога или текущего каталога по умолчанию; `envIRON` — для вывода всех переменных среды; `echo` — для вывода текста на экран; `help` — для отображения руководства пользователя; `pause` — для приостановки работы до нажатия клавиши `Enter`; `quit` — для завершения работы оболочки.

2. Описание реализации

Для реализации командной оболочки `myshell` были подготовлены два исходных файла на языке C: `myshell.c`, где находится основная логика оболочки и цикл обработки команд, и `utility.c`, в котором реализованы встроенные команды и вспомогательные функции. Также используется заголовочный файл `myshell.h`, содержащий объявления структур данных, констант и прототипов функций. Сборка проекта выполняется с помощью файла `makefile`, который компилирует указанные исходники в исполняемый файл `myshell`.

Программа разделена на три основных файла: `myshell.c` отвечает за основной цикл работы оболочки, `utility.c` содержит реализацию встроенных команд, а `myshell.h` хранит объявления констант, подключаемых библиотек и прототипов функций. Такое разделение делает проект более удобным для сопровождения, так как логика запуска команд и вспомогательные функции вынесены отдельно.

В заголовочном файле определены основные ограничения, например размер буфера ввода, как представлено на рис. 2.1. Это позволяет унифицировать обработку команд и избежать дублирования констант в разных частях программы.

```
#define MAX_BUFFER 1024
#define MAX_ARGS 64
#define SEPARATORS " \t\n"
```

Рис 2.1 Основные ограничения в заголовочном файле

Файл `myshell.c` реализует функцию `main`, которая запускает оболочку и обрабатывает ввод пользователя. В начале определяется путь к самому исполняемому файлу с помощью `realpath(argv[0], shell_path)`, после чего этот путь сохраняется в переменной среды `shell` через `setenv("shell", shell_path, 1)`. Это полезно, если программе нужно передавать информацию о своём расположении или использовать её при запуске дочерних процессов.

Основной ввод осуществляется либо из стандартного потока `stdin`, либо из `batch`-файла, если имя файла передано аргументом командной строки. Например, при наличии файла входа используется конструкция: `input = fopen(argv[1], "r");`. Если открыть файл не удаётся, программа выводит сообщение об ошибке и завершает работу рис.2.

```
FILE *input = stdin;
if (argc > 1) {
    input = fopen(argv[1], "r");
    if (!input) {
        perror("myshell: batch file");
        return 1;
    }
}
```

рис 2.2 Вывод сообщения об ошибке

Встроенная команда `cd` реализована в `internal_cd` рис.3 и используется для смены текущего каталога. Если аргумент не указан, функция выводит текущий путь через `getcwd`; если аргумент есть, выполняется переход

командой `chdir(args[1])`. После успешного перехода обновляется переменная среды `PWD` с помощью `setenv("PWD", cwd, 1)`.

```
void internal_cd(char **args) {
    if (args[1] == NULL) {
        char cwd[PATH_MAX];
        if (getcwd(cwd, sizeof(cwd)) != NULL) {
            printf("%s\n", cwd);
        }
    } else {
        if (chdir(args[1]) != 0) {
            perror("myshell: cd");
        } else {
            char cwd[PATH_MAX];
            getcwd(cwd, sizeof(cwd));
            setenv("PWD", cwd, 1);
        }
    }
}
```

Рис 2.3 реализация команды `cd`

Команда `dir` выводит содержимое каталога. Если путь не указан, по умолчанию используется текущий каталог `..`. Пример: `char *path = (args[1] == NULL) ? "." : args[1];`. Затем каталог открывается через `opendir`, а его содержимое читается в цикле `readdir` и выводится на экран.

Команда `environ` выводит все переменные окружения рис.4. Для этого используется глобальный массив `environ`, и каждая переменная печатается по очереди: `for (char **env = environ; *env != 0; env++)`. Команда `echo` просто выводит все переданные слова в одной строке, разделяя их пробелами.

```
void internal_environ() {
    for (char **env = environ; *env != 0; env++) {
        printf("%s\n", *env);
    }
}
```

Рис 2.4 реализация `environ`

Команда `help` выводит краткое руководство пользователя, где перечислены все встроенные команды и их назначение. Внутри функции

используется несколько вызовов `printf`, формирующих текст справки. Это делает оболочку более удобной для пользователя, поскольку он может быстро узнать список доступных операций.

Команда `pause` приостанавливает выполнение оболочки до нажатия клавиши `Enter`. Реализация проста: программа выводит приглашение `Press Enter to continue...`, а затем ждёт ввода символа новой строки в цикле `while (getchar() != '\n');`.

Запуск внешних программ

Если команда не относится к встроенным, оболочка пытается запустить внешнюю программу. Для этого создаётся дочерний процесс через `fork()`. В дочернем процессе вызывается `execvp(args[0], args)`, который заменяет текущий образ процесса на указанную программу.

Перед запуском внешней команды дочерний процесс дополнительно устанавливает переменную среды `parent`, в которой хранится путь к оболочке. Если `execvp` завершается ошибкой, выводится сообщение через `perror("myshell")`, после чего процесс завершается с `EXIT_FAILURE`. Родительский процесс, в свою очередь, ждёт завершения дочернего через `waitpid(pid, NULL, 0)`.

3. Тестирование

На рисунках 3.1-3.7 представлены тесты разработанной оболочки.

```
myshell> cd  
/home/danil/myshell-lab  
myshell>
```

Рис 3.1. Тест внутренней команды `cd`

```
myshell> dir  
myshell.h myshell . .. makefile test.txt myshell.c utility.c readme .vscode
```

Рис 3.2. Тест внутренней команды dir

```
myshell> environ
SHELL=/bin/bash
COLORTERM=truecolor
WSL2_GUI_APPS_ENABLED=1
TERM_PROGRAM_VERSION=1.117.0
WSL_DISTRO_NAME=Ubuntu-24.04
NAME=WIN-CUBCPHMT5SL
PWD=/home/danil/myshell-lab
LOGNAME=danil
VSCODE_GIT_ASKPASS_NODE=/home/danil/.vscode
HOME=/home/danil
LANG=C.UTF-8
WSL_INTEROP=/run/WSL/439_interop
```

Рис 3.3. Тест внутренней команды environ

```
myshell> pwd
/home/danil/myshell-lab
```

Рис 3.4. Тест внешней команды pwd

```
myshell> echo hello
hello
```

Рис 3.5. Тест внутренней команды echo

```
myshell> pause
Press Enter to continue...
```

Рис 3.6. Тест внутренней команды pause

```
myshell> quit
danil@WIN-CUBCPHMT5SL:~/myshell-lab$
```

Рис 3.7. Тест внутренней команды quit

```
danil@WIN-CUBCPHMT5SL:~/myshell-lab$ ./myshell test.txt
Привет из файла

--- Руководство пользователя myshell ---
cd <directory>      : Смена текущего каталога. Без аргументов выводит текущий путь.
clr                 : Очистка экрана терминала.
dir <directory>     : Вывод содержимого указанного каталога.
environ            : Вывод всех переменных окружения.
echo <comment>      : Вывод сообщения на экран.
help               : Вывод этого справочного руководства.
pause              : Приостановка работы оболочки до нажатия клавиши Enter.
quit               : Завершение работы и выход из оболочки.
-----
Press Enter to continue...
```

Рис 3.8 Работа пакетное режима

4. Вывод

В ходе выполнения лабораторной работы разработана командная оболочка `myshell`, реализующая восемь встроенных команд, логику запуска внешних программ через `fork()` и `exec()` с передачей переменной окружения `parent`, а также пакетный режим чтения команд из файла.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Керниган Б. У. Язык программирования C / Б. У. Керниган, Д. М. Ричи. - 2-е изд. - М. : Вильямс, 2021. - 304 с.
2. Робачевский А. М. Операционная система UNIX / А. М. Робачевский. - 2-е изд. - СПб. : БХВ-Петербург, 2010. - 656 с.